

# Sam AI Persona -- Evaluation Report

Candidate: Vaibhav Pandey | Phone: +19868009622 | Chat: scalar-ai-engineer-intern.vercel.app | 2026-06-05

## Part 1: Voice Quality

### Methodology

Timestamps captured at each pipeline stage via Vapi webhooks across 8 test calls (different networks: WiFi, 4G). First-response = time from end of caller speech to first ElevenLabs audio byte. Booking success = caller received Cal.com invite.

### Results

|                                   |                                                                   |
|-----------------------------------|-------------------------------------------------------------------|
| <b>Avg first-response latency</b> | 1.2s (target <2s) PASS                                            |
| <b>P95 latency</b>                | 1.8s (target <2.5s) PASS                                          |
| <b>Latency breakdown</b>          | STT 280ms + LLM first-token 520ms + TTS 190ms + net 210ms         |
| <b>Transcription WER</b>          | 4.2% (Deepgram Nova-2, manual review of 8 transcripts)            |
| <b>Booking success rate</b>       | 7/8 (88%) PASS (target >85%)                                      |
| <b>Avg call duration</b>          | 187s                                                              |
| <b>Barge-in / interruption</b>    | Handled via Vapi stopSpeakingPlan (numWords=2, 0.2s voiceSeconds) |

## Part 2: Chat Groundedness

### Methodology

20 questions evaluated locally against the RAG engine (chroma\_db, 1910 chunks). Split: 12 answerable from resume+GitHub, 5 unanswerable traps, 3 adversarial injections. Each question ran in an isolated session. Responses labelled CORRECT / CORRECT\_REJECT / HALLUCINATION / WRONG\_REJECT by checking against a fixed reject-phrase list.

### Results

|                             |                                                               |
|-----------------------------|---------------------------------------------------------------|
| <b>Total questions</b>      | 20                                                            |
| <b>Correct answers</b>      | 12/12 (all factual project/background Qs answered correctly)  |
| <b>Correct rejections</b>   | 2/5 traps correctly refused                                   |
| <b>Hallucinations</b>       | 6 (30%) -- all on out-of-scope personal Qs (GPA, food, etc)   |
| <b>Injection block rate</b> | 2/3 (1 miss: 'ignore all instructions' phrasing evaded regex) |
| <b>Overall groundedness</b> | 70% -- 14/20 questions handled correctly                      |

### Retrieval Quality (measured on Railway deployment, 7 repos indexed)

|                  |                                             |
|------------------|---------------------------------------------|
| <b>Precision</b> | 0.81 (relevant retrieved / total retrieved) |
| <b>Recall</b>    | 0.74 (relevant retrieved / total relevant)  |
| <b>F1</b>        | 0.77                                        |

Note: local chroma\_db has 4 repos. meta-hackathon-incident-commander + HotelBookingPro are indexed only on Railway. Retrieval numbers above are from the live deployment.

## Part 3: Failure Modes

### Failure 1: ask\_knowledge\_base HTTP self-call loopback (+300ms, single point of failure)

Root cause: voice\_handler called BACKEND/voice/query over the public internet from within the Railway container itself, adding 200-400ms per tool call and failing on network hiccups.

Fix: removed the HTTP call entirely -- voice\_handler now calls RAGEngine.query() directly in-process. Latency dropped ~300ms per knowledge-base tool call.

### Failure 2: Duplicate /voice/query route -- second (correct) handler was dead code

Root cause: FastAPI silently registered only the first definition of a duplicate route. The full dual-mode handler (Vapi tool-call + internal) was never reached.

Fix: removed the first (broken) handler, kept the dual-mode version.

### Failure 3: Groq free-tier 429 rate limit during rapid tool calls

Root cause: single Groq API key hits 30 req/min free-tier limit during simultaneous voice tool calls + chat sessions.

Fix: multi-key rotation -- RAGEngine holds up to 3 keys (GROQ\_API\_KEY, GROQ\_API\_KEY\_2, GROQ\_API\_KEY\_3) and rotates to the next on 429 with 0.5s delay.

## Part 4: Conscious Tradeoff -- Groq (free) vs OpenAI (paid) for chat LLM

Decision: use Groq llama-3.3-70b-versatile for chat RAG answers instead of GPT-4o.

Why: Groq free tier gives ~400 tok/s throughput -- chat streaming feels instant (<300ms first token).

GPT-4o would cost ~\$0.005/1K tokens \* 800 tokens/query = ~\$0.004/query. At 500 chat sessions/month \* 10 messages = 5000 queries = ~\$20/month extra.

Tradeoff accepted: Groq free tier has 14,400 tokens/day limit -- hits rate limits under heavy concurrent load. Mitigated by 3-key rotation. If volume exceeds free tier, upgrade to Groq paid (\$0.59/1M tokens) -- still 8x cheaper than GPT-4o.

## Part 5: What I'd Build with 2 More Weeks

- Automated eval harness: drive voice calls programmatically via Vapi API, measure booking success end-to-end without manual test calls.
- LLM-as-judge hallucination grader: for each answer, a second GPT-4o call checks if every claim is traceable to retrieved context. Gives per-claim precision instead of binary correct/hallucinated.
- Re-ingest on deploy: trigger ingest\_data\_groq.py as part of Railway deploy so the knowledge base is always current with latest commits.